

UNIVERSIDAD DE LAS AMÉRICAS PUEBLA
DEPARTAMENTO DE COMPUTACIÓN, ELECTRÓNICA Y
MECATRÓNICA



Proyecto - F1

FORMULA CHEMS

Programación Orientada a Objetos (POO)

Presentado por:

Amy Marianee Ramírez Sánchez
Juan Pablo López Moreno
Jose Maria Blancarte López

22 de junio de 2026

Índice

1. Contexto	2
2. Motivación	2
3. Objetivos	2
4. Enfoque	3
5. Fase de Análisis	3
6. Fase de Diseño	4
7. Implementación	5
7.1. Entidades Básicas	5
7.2. Motor de Simulación	6
7.3. El Flujo y Control de Datos	6
7.4. Director de Orquesta	6
8. Anexos	6
8.1. Justificación SO2	6
8.2. Códigos para S02	7
8.2.1. Identify	7
8.2.2. Analyze	8
8.2.3. Evaluate Solutions	8
8.2.4. Develop Solutions	9
8.2.5. Implement Engineering Design	9
8.2.6. Consider non-technical issues	10
8.3. Plan de Trabajo Interno	11
8.4. Registro de Reuniones Internas	11
8.5. Porcentaje de Participación	11
8.6. Metodología de Implementación	11
9. Referencias	13

1. Contexto

Llamada la competencia por excelencia del automovilismo, la Fórmula 1 (F1) es una serie de carreras que impulsan al límite la tecnología, la ciencia y las construcciones que participan en ella.

Esta competencia está compuesta por una variedad de elementos, entre los cuales se encuentran temporadas de carreras denominadas "Gran Premio", la culminación de 3 clasificatorias y más de 2 docenas de pilotos compitiendo por los mejores tiempos.

2. Motivación

La Fórmula 1 es el deporte de autos más popular del mundo, donde cada segundo cuenta y se genera una cantidad enorme de datos en cada carrera. Lo que nos motiva a desarrollar este proyecto es el reto de tomar toda esa información (equipos, pilotos, tiempos y resultados) y ordenarla en un sistema fácil de usar.

Queremos diseñar una herramienta sencilla que limpie la complejidad de los datos, permitiendo a cualquier aficionado entender los resultados y revivir la emoción de cualquier temporada de forma clara.

3. Objetivos

Desarrollar un sistema orientado a objetos para registrar y administrar la información de las temporadas de Fórmula 1, con la capacidad de almacenar de forma independiente y reproducir el historial estadístico de cualquier año de competición.

Objetivos específicos:

- **Administrar escuderías y recursos técnicos:** Orquestrar la creación de equipos deportivos aplicando restricciones reglamentarias estrictas (límites operativos de pilotos), y parametrizar las características mecánicas de los monoplazas (motor, llantas y peso en escalas validadas) junto con el factor de habilidad de cada conductor.
- **Calendarizar y configurar Grandes Premios:** Estructurar la temporada de competición definiendo el nombre, nivel de dificultad y las ventanas de tiempos históricos (mínimos y máximos) para cada circuito, permitiendo el control del progreso cronológico del año.
- **Registrar clasificaciones mediante simulación probabilística:** Procesar los tiempos de las sesiones Q1, Q2 y Q3 utilizando funciones de distribución gaussiana para modelar estadísticamente la variabilidad de la pista, y aplicar un motor lógico de filtrado que asigne posiciones iterativamente hasta ensamblar la parrilla de salida oficial.
- **Controlar resultados y campeonatos globales:** Simular la carrera dominical ponderando la habilidad del conductor, la eficiencia del vehículo y factores de aleatoriedad para el cálculo de rebases, documentando las posiciones finales y actualizando automáticamente el estado del campeonato mundial.

- **Generar reportes visuales y persistencia de datos:** Producir de forma automatizada tablas gráficas e indicadores de desempeño accesibles para el usuario, asegurando la trazabilidad de la simulación mediante la exportación y lectura estructurada de archivos en formato JSON.

4. Enfoque

Basándose en esta necesidad y en la pasión compartida por un deporte tan tecnológicamente complejo, avanzado e innovador. El proyecto se centra en responder una pregunta clave: ¿Es posible administrar y recrear los resultados de la Fórmula 1 usando el paradigma de programación orientada a objetos?

Siguiendo esta pregunta, y convencidos de sus posibles soluciones, presentamos este documento como reporte de las propuestas, desarrollos e implementaciones de esta solución.

5. Fase de Análisis

Debido a las particulares necesidades del problema consideramos este flujo básico para poder cumplir con las relaciones. Lo que se busca con las relaciones establecidas en el flujo es permitirnos adaptar a cualquier cantidad de escuderías y pilotos por cada temporada.

Las relaciones entre los objetos se establecieron siguiendo un flujo de dependencias lógicas muy claro: la temporada rige las carreras y las escuderías; las escuderías agrupan los recursos operativos como los vehículos y los pilotos; y las carreras controlan la ejecución de las sesiones competitivas.

El vehículo y el piloto se diseñaron como entidades separadas pero vinculadas a la escudería, lo que otorga la flexibilidad necesaria para modificar las características de un auto o los datos de un conductor sin alterar la estructura general del equipo. Por otro lado, la bifurcación de la carrera en sesiones de clasificación divididas en etapas y una carrera final permite que los tiempos medidos en las primeras fases sirvan como el criterio de ordenamiento para la parrilla de salida, reflejando fielmente las reglas de la competencia.

Con este planteamiento, se abarca la gran mayoría de los requerimientos solicitados en esta primera etapa, ya que el modelo es capaz de almacenar con precisión los datos logísticos de los premios, el inventario técnico de los equipos y los resultados de las posiciones alcanzadas tanto en pruebas como en el Gran Premio definitivo. Es fundamental destacar que el diseño actual constituye un avance inicial y una base sólida para el proyecto, enfocado primordialmente en la correcta administración, almacenamiento y consistencia de los datos históricos de la Fórmula 1. Al tener esta estructura de herencia y asociaciones bien definida, el sistema queda completamente preparado para escalar en fases posteriores del desarrollo.

Más adelante, se planifica trabajar con la implementación de algoritmos avanzados y lógica de programación matemática que aprovechen atributos clave como el nivel de habilidad del piloto, las características de los neumáticos o el rendimiento del motor. Esto permitirá realizar cálculos complejos, simulaciones de carreras en tiempo real basadas en probabilidades y la predicción de resultados competitivos con un nivel

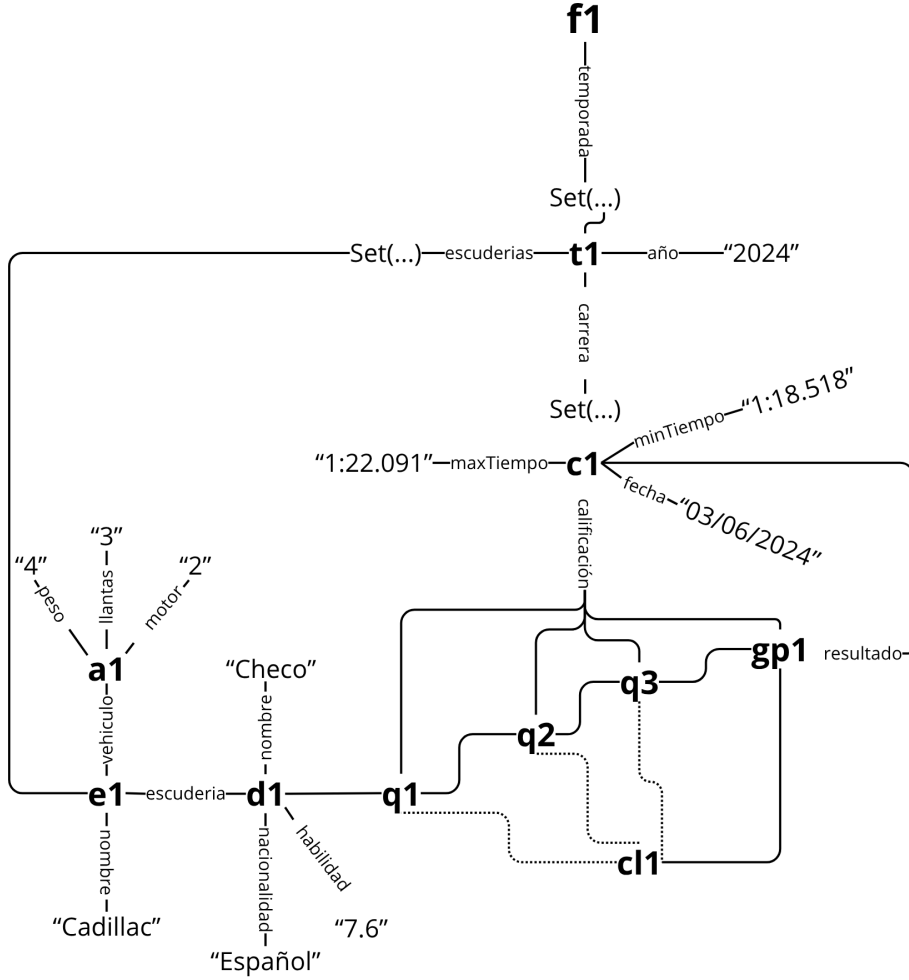


Figura 1: Diagrama de relaciones lógicas F1

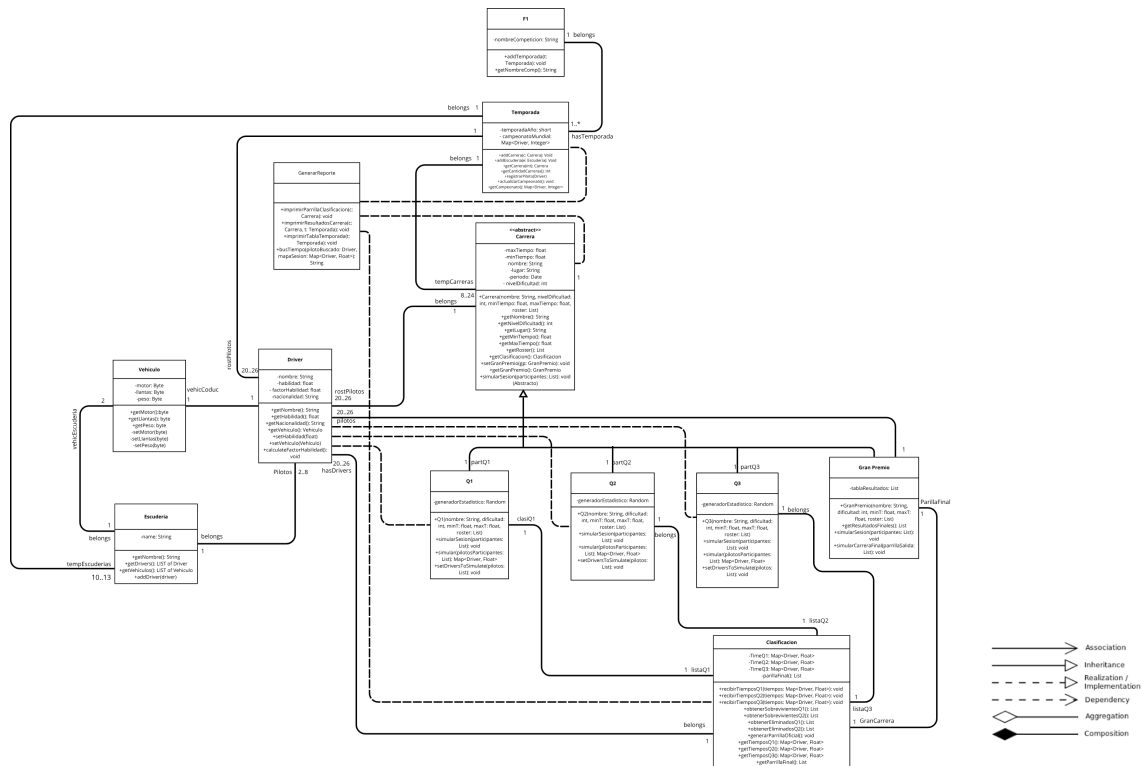
de precisión mucho mayor, transformando el sistema de una base de datos estática a un entorno dinámico de simulación deportiva.

6. Fase de Diseño

En la fase de diseño el equipo se enfocó en mejorar las propuestas, optimizar el desarrollo y completar todos los objetivos. Al modificar el planteamiento inicial, ampliamos varios métodos y clases para poder implementar de manera ordenada y más orientada a objetos, estableciendo el flujo y relaciones existentes entre las diversas clases.

El desafío principal de esta sección radica en establecer las relaciones apropiadas para hacer el mayor uso de todos los atributos y clases, sobre todo los vínculos entre las clases como Carrera y sus dependientes como Q1, Q2 y Q3, cuyas clases también tienen un vínculo de dependencia con Clasificación para poder procesar las listas adecuadas.

Asimismo, establecer un orden apropiado para distribuir y aprovechar ciertos comportamientos tenemos la clase Tabla la cual tiene interacciones con Carrera, Clasificación y Temporada. Todos estos cambios a las bases principales, al igual que



7.2. Motor de Simulación

- **Carrera:** La clase de carrera nos ayuda a definir el nombre de la pista, su nivel de dificultad y los tiempos máximos y mínimos históricos (para poder simular los tiempos dentro de esto). Asimismo, esta clase contiene un método abstracto `simularSesion()` el cual en clases de qualis (Q1, Q2, Q3) se fuerce a simular según se dicte en cada clase.
- **Q1, Q2, Q3:** Estas clases heredan todo de Carrera y ocupan sus propias fórmulas matemáticas para poder realizar el cálculo y lista de los tiempos de los pilotos. El equipo implementó una campana gaussiana para poder determinar la complejidad de cada Qualy, con diferentes factores (1.2, 1 y 0.8) para determinar la desviación estándar de los eventos.
- **Gran Premio:** Simula el evento principal de los domingos, y basado en probabilidades establece mediante la parrilla de inicio los resultados finales de cada carrera. Esto mediante la implementación de un factor de aleatoriedad, que por cada piloto adyacente, se genera un número decimal pseudo aleatorio entre 0.0 y 1.0 usando `Math.random()`, para después establecer la condición `Math.random() > 0.75` que instituye a cualquier piloto que caiga dentro de esta condición un intercambio de posición (simulando el rebase). Guardando la lista modificada como tabla de posición final al terminar el recorrido.

7.3. El Flujo y Control de Datos

- **Clasificación:** Nos ayuda a guardar al piloto con su tiempo y mediante Stream de Java, nos permite establecer quienes pasan al siguiente Qualy, asimismo, nos apoya creando la parrilla a partir de los eliminados y los resultados de todos los clasificatorios.
- **Temporada:** El equipo desarrolló una clase que nos permite analizar el año, escuderías, carreras y puntos recopilados durante cada temporada. Asimismo, nos permite resimular una temporada, borrando la tabla de puntos y simulado desde la carrera 1 de la temporada.

7.4. Director de Orquesta

- **Main:** En esta clase de prueba configuramos los 22 vehículos y los 22 pilotos con los datos reales de la temporada 2026. Las 11 escuderías existentes y 24 carreras para la temporada. Igual se configuró un switch básico para poder realizar acciones y llamar métodos.

8. Anexos

8.1. Justificación SO2

- **Identify:** El proyecto se enfoca en la vida real, estableciendo las regulaciones y limitaciones de acuerdo a los reglamentos y estándares establecidos por la FIA y las empresas detrás de la carrera. Esto se refleja, por ejemplo, en la clase de **Escudería**, con las limitantes del número de pilotos y vehículos.

- **Analyze:** Para simular una variabilidad estadística se implementó una campana gaussiana con ciertas desviaciones establecidas en cada Qualifier, estos pasados en el comportamiento real de las carreras y resultados, basado en los tiempos históricos.
- **Evaluate solutions:** Un enfoque principal del proyecto fue asegurar la comparación y facilidad de acceso a los datos simulador para su comparativa a futuro.
- **Develop Solutions:** Varios intentos se realizaron para poder realizar de mejor manera las simulaciones y llegar al objetivo final. Por esta misma razón se ocupó el archivo *Manejador de Archivos*, para mejorar el proyecto basado con alternativas tecnológicas.
- **Implement Engineering Design:** El proyecto al implementarse se considera 100 % funcional, escalando su funcionamiento mediante una interfaz gráfica, la lógica se ensambla de manera completa de manera jerárquica.
- **Consider non-techincal issues:** EL proyecto se desarrollo con un enfoque basado en fans de la Fórmula 1, buscando una simplicidad y facilidad de acceso con los usuarios. Considerando todas las posibilidades y aceptando la mejora y desarrollo según las necesidades de los usuarios.

8.2. Códigos para S02

8.2.1. Identify

```

1 // Fragmento de Vehiculo.java demostrando restricciones de ingeniería
  (0 a 10)
2 public void setMotor(byte mo) {
3     if (mo < 0 || mo > 10) {
4         throw new IllegalArgumentException("La característica debe
5         estar entre 0 y 10. Valor intentado: " + mo);
6     }
7     this.motor = mo;
8 }
9 // Fragmento de Escuderia.java demostrando restricciones logísticas
10 private static final byte MIN_PILOTOS = 2;
11 private static final byte MAX_PILOTOS = 8;
12
13 public Escuderia(String n, Vehiculo v1, Vehiculo v2, List<Driver>
14     drivers){
15     if(drivers.size() < MIN_PILOTOS || drivers.size() > MAX_PILOTOS){
16         throw new IllegalArgumentException("La lista de pilotos debe
17         tener entre " + MIN_PILOTOS + " y " + MAX_PILOTOS + " para ser
18         valida.");
19     }
20     this.nombre = n;
21     // ...
22 }

```

Listing 1: Restricciones y estándares en Vehículo y Escudería

8.2.2. Analyze

```
1  /*
2  * Ecuación única centralizada para simular tiempos de clasificación.
3  * Utiliza los bonos de habilidad e ingeniería de cada piloto,
4  * aplicando la
5  * desviación estándar configurada de manera particular por cada
6  * subclase.
7  */
8  public Map<Driver, Float> simularFase(List<Driver> pilotosParticipantes
9  ) {
10     Map<Driver, Float> resultados = new HashMap<>();
11
12     for (Driver d : pilotosParticipantes) {
13         float bonoHabilidad = d.getHabilidad() * 2.5f;
14         float bonoVehiculo = (d.getVehiculo().getMotor() / 10.0f) * 2.5
15         f;
16
17         float media = maxTiempo - bonoHabilidad - bonoVehiculo;
18
19         // Generación del tiempo usando la desviación de la sesión
20         actual
21         float tiempo = media + ((float) rng.nextGaussian() *
22         desviacionEstandar);
23
24         if (tiempo < minTiempo) {
25             tiempo = minTiempo + (rng.nextFloat() * 0.2f);
26         }
27         resultados.put(d, tiempo);
28     }
29     return resultados;
30 }
```

Listing 2: Análisis estadístico en Carrera.java

8.2.3. Evaluate Solutions

```
1  // Fragmento de Clasification.java: Motor lógico de filtrado
2  /*
3  * Fórmula dinámica: calcula cuántos pilotos quedan fuera por ronda
4  * basada en el total de inscritos (N).
5  */
6  private int calcularEliminados() {
7      if (totalPilotos == 0) return 5; // fallback seguro
8      return (totalPilotos - 10) / 2;
9  }
10
11 // Fragmento de VentanaTabla.java: Evaluación de pre-condiciones
12 // visuales
13 public void actualizarDatos(Object[][] filas, List<Driver> ordenFilas,
14 Temporada temporada) {
15     modelo.setRowCount(0);
16     for (Object[] fila : filas) {
17         modelo.addRow(fila);
18     }
19
20     if (filas.length == 0) {
```

```

19         JOptionPane.showMessageDialog(this,
20             "Todavía no hay datos para mostrar en esta tabla.\n" +
21             "Simula la sesión correspondiente primero.",
22             "Sin datos", JOptionPane.INFORMATION_MESSAGE);
23     }
24 }

```

Listing 3: Evaluación lógica en Clasification.java y VentanaTabla.java

8.2.4. Develop Solutions

```

1 // --- GUARDAR PROGRESO ---
2 public static void guardarProgresoCarrerasJson(Temporada t) {
3     String nombreArchivo = t.getAño() + "_progreso.json";
4     StringBuilder sb = new StringBuilder();
5     sb.append("{\n  \"año_temporada\": ").append(t.getAño()).append(",\n  \"carreras\": [\n");
6
7     for (int i = 0; i < t.getCarrera().size(); i++) {
8         Carrera c = t.getCarrera().get(i);
9         sb.append("    { \"ronda\": ").append(i + 1)
10            .append(", \"nombre_circuito\": \"").append(c.getNombre()).
11            append("\n")
12            .append(", \"dificultad\": ").append(c.getNivelDificultad()).
13            append(" }");
14         if (i < t.getCarrera().size() - 1) sb.append(",");
15         sb.append("\n");
16     }
17     sb.append("  ]\n}");
18     escribirTextoEnDisco(nombreArchivo, sb.toString());
19 }

```

Listing 4: Metodología de serialización en ManejadorArchivos.java

8.2.5. Implement Engineering Design

```

1 /*
2  * Embudo de simulación: Q1 -> Q2 -> Q3 -> Gran Premio.
3  */
4 public void simularFinDeSemana(Carrera c) {
5     Q1 q1 = new Q1(c.getNombre(), c.getNivelDificultad(), c.
6         getMinTiempo(), c.getMaxTiempo(), c.getRoster());
7     Q2 q2 = new Q2(c.getNombre(), c.getNivelDificultad(), c.
8         getMinTiempo(), c.getMaxTiempo(), c.getRoster());
9     Q3 q3 = new Q3(c.getNombre(), c.getNivelDificultad(), c.
10        getMinTiempo(), c.getMaxTiempo(), c.getRoster());
11     GranPremio gp = new GranPremio(c.getNombre(), c.getNivelDificultad(),
12        c.getMinTiempo(), c.getMaxTiempo(), c.getRoster());
13
14     c.setGranPremio(gp);
15     Clasification juez = c.getClasificacion();
16
17     juez.recibirTiemposQ1(q1.simular(c.getRoster()));
18     List<Driver> pasanQ2 = juez.obtenerSobrevivientesQ1();
19
20     juez.recibirTiemposQ2(q2.simular(pasanQ2));
21 }

```

```

17     List<Driver> pasanQ3 = juez.obtenerSobrevivientesQ2();
18
19     juez.recibirTiemposQ3(q3.simular(pasanQ3));
20     juez.generarParrillaOficial();
21
22     gp.simularCarreraFinal(juez.getParrillaFinal());
23     temporada.actualizarCampeonato();
24 }

```

Listing 5: Integración del sistema en SimuladorController.java

8.2.6. Consider non-technical issues

```

1  // Fragmento de Driver.java: Manejo de diversidad cultural
2  public class Driver {
3      private String nombre;
4      private String nacionalidad; // Código internacional de
5      nacionalidad (ej: MEX)
6      // ...
7  }
8
9  // Fragmento de PanelDesempeno.java: Accesibilidad gráfica para el
10 usuario final
11 @Override
12 protected void paintComponent(Graphics g) {
13     super.paintComponent(g);
14     Graphics2D g2 = (Graphics2D) g;
15     g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints
16     .VALUE_ANTIALIAS_ON);
17
18     // Renderizado dinámico adaptado a la lectura humana
19     for (int i = 0; i < etiquetas.length; i++) {
20         int valor = valores[i];
21         int anchoBarra = (int) ((valor / (double) maxValor) *
22         anchoDisponible);
23         Color color = (colores != null && i < colores.length && colores
24         [i] != null)
25             ? colores[i] : new Color(120, 120, 120);
26
27         // Barra visual
28         g2.setColor(color);
29         g2.fillRoundRect(MARGEN_IZQ, y, Math.max(anchoBarra, 4),
30         ALTO_BARRA, 6, 6);
31
32         // Valor numérico explícito
33         g2.setColor(Color.BLACK);
34         g2.drawString(String.valueOf(valor), MARGEN_IZQ +
35         anchoDisponible + 10, yTextoBase);
36         // ...
37     }
38 }

```

Listing 6: Consideraciones socioculturales y usabilidad

8.3. Plan de Trabajo Interno

El proyecto se estructuró en tres fases secuenciales, distribuyendo las responsabilidades principales de la siguiente manera (todos apoyamos en cada fase, solo que se distribuyó con un líder por actividad):

- **Fase 1: Investigación y Recopilación**

- Responsable principal: Juan Pablo Lopez Moreno
- Actividades: Búsqueda de fuentes y estructuración de las necesidades para el grafo.

- **Fase 2: Desarrollo y Análisis**

- Responsable principal: Jose Maria Blancarte Lopez
- Actividades: Procesamiento de la información, revisión del grafo, creación de las bases del UML.

- **Fase 3: Revisión y Edición Final**

- Responsable principal: Amy Marianne Ramírez Sánchez
- Actividades: Corrección de UML, verificación de formato, revisado de documento completo.

8.4. Registro de Reuniones Internas

Para garantizar la alineación del equipo, se llevaron a cabo sesiones de coordinación a través de medios virtuales:

8.5. Porcentaje de Participación

La carga de trabajo se distribuyó de manera equitativa. Cada integrante asumió el liderazgo de una fase y colaboró activamente en la revisión de los aportes de los demás.

8.6. Metodología de Implementación

1. Traducción del UML a código:

- Objetivo: Trasladar el diseño conceptual estructurado en la Fase 2 a un entorno de desarrollo.
- Acciones: Se crearon los archivos y se declararon las clases principales definidas en el diagrama UML.

2. Desarrollo fundamental del método:

- Objetivo: Interconectar las clases.
- Acciones: Cada integrante tomó la responsabilidad de desarrollar la lógica interna de métodos específicos. Se programaron las interacciones entre los objetos del sistema de F1 para que funcionaran de manera sincronizada.

Fecha	Objetivo de la Reunión	Asistentes	Acuerdos Clave
07/05/2026	Sesión de Inicio (Kick-off): Definición del enfoque del reporte y asignación de roles.	Todos	Se aprobó el cronograma y la división de tareas.
08/05/2026	Revisión de Avances: Puesta en común de la información recopilada y primeros borradores.	Todos	Se reajustaron las clases por redundancia y/o falta de métodos.
11/06/2026	Cierre y Validación: Lectura final del documento y control de calidad.	Todos	Se aprobó la versión final para su entrega.
12/06/2026 - 14/06/2026	Sesión de feedback y mejora del diseño, ampliación y corrección de diagramas. Revisión de Creación de estructuras bases y relaciones de las clases.	Todos	Se aprobó el cronograma y la división de tareas. Se reajustaron las clases por redundancia y/o falta de métodos.
16/06/2026	Cierre y Validación: Lectura final del documento, pruebas en main y revisión de objetivos de la entrega.	Todos	Se aprobó la versión final para su entrega.

3. Integración, Pruebas y Debugging:

- Objetivo: Asegurar el correcto funcionamiento mencionado en la fase 2.
- Acciones: Se ensamblan todos los módulos en un solo flujo de ejecución. Se realizaron pruebas para rastrear errores y se ajustaron aquellas clases que presentaron redundancia.

Integrante	Rol Principal	Porcentaje de Participación
Juan Pablo Lopez Moreno	Gestor de Información e Investigador	33.3 %
Jose Maria Blancarte Lopez	Redactor y Analista Principal	33.3 %
Amy Marianee Ramirez Sanchez	Editor y Control de Calidad	33.4 %
Total		100 %

9. Referencias

- All the information you need about the 2026 FI regulations. (n.d.). <https://www.formula1.com/en/page/2026-fl-regulations>
- What is F1? Formula 1 Explained - Drivers, Teams, Calendar, Grand Prix, Sprint, Qualifying & Scoring. (n.d.). <https://www.formula1.com/en/page/what-is-fl>